

Supplementary Technical Details for DecouplePay

OVERVIEW

This document provides supplementary technical details for *DecouplePay: Routing-Friendly Payer-Payee Unlinkability for Bitcoin Payment Channel Networks*. It complements the main paper by presenting construction-level details that are useful for understanding the concrete realization of DecouplePay but are omitted from the main presentation for space and readability.

The document focuses on two components of the construction. First, we revisit the Schnorr-based Anonymous Multi-Hop Lock (AMHL) mechanism used on the payer-to-final-intermediary routed path. We make explicit the algebraic structure of its setup, lock generation, verification, and backward release procedures, and clarify how the path-dependent state is embedded into an incomplete Schnorr response. Second, we illustrate the evolution of the persistent payment channel between the final intermediary and the payee, including the dedicated redemption output, successive channel states, time-out handling, and revocation of obsolete states.

These two components serve different purposes in DecouplePay. Schnorr-AMHL provides path-dependent release and wormhole resistance on the routed payment path, whereas the final-intermediary/payee channel provides the enforceable payee-side redemption mechanism. They are connected by the delayed blind-signature response used by DecouplePay: a successful routed release reveals the response needed by the payer to derive the payee-side redemption authorization.

DOCUMENT ORGANIZATION

Section I revisits the Schnorr-based AMHL construction used on the routed payment path. It describes the path-dependent setup state, lock generation, the distinction between an incomplete Schnorr response and a valid opening key, the verification rule, and the backward release transformation. We additionally explain how this mechanism is used together with the independent blind-signature-response condition in DecouplePay.

Section II illustrates the evolution of the payment channel between the final intermediary F and the payee P . It shows how a fixed payment value is reserved in a dedicated redemption output, how the corresponding claim is resolved, and how successive channel states revoke obsolete redemption descriptors.

The complete DecouplePay protocol, leakage model, UC security analysis, generic construction, Taproot/P2TR-compatible illustration, and performance evaluation are provided in the main paper. This document focuses only on the additional algebraic and channel-state details needed to understand the concrete construction.

I. SCHNORR-BASED AMHL

This section revisits the Schnorr-based anonymous multi-hop lock (AMHL) construction [1] used by DecouplePay on

the payer-to-final-intermediary routed path. Rather than reproducing the complete security treatment of AMHL, we focus on the algebraic structure needed to understand its use in DecouplePay.

The key idea is that each routed lock contains an incomplete Schnorr response whose completion depends on path-specific secret information. A downstream release reveals enough information for an intermediary to complete the opening of its incoming lock, thereby enforcing backward, path-dependent release. This mechanism provides the wormhole-resistance property required by the routed component of DecouplePay.

Let $\mathbb{G}$ be a cyclic group of prime order q with generator G . For a Schnorr public key pk , a signature (R, s) on message m is verified by computing $e = H(\text{pk} \parallel R \parallel m)$ and checking $sG = R + e\text{pk}$. We use this sign convention throughout this section.

A. Setup

For a path $(U_0, \dots, U_n)$, the AMHL setup samples $y_0, \dots, y_{n-1} \leftarrow \mathbb{Z}_q$ and defines $Y_i = \left(\sum_{j=0}^i y_j\right)G$. Hence, for every intermediate node, $Y_i = Y_{i-1} + y_iG$.

Node U_i receives the path-dependent state $s_i^I = (Y_{i-1}, Y_i, y_i)$, while the receiver obtains $z_{n-1} = \sum_{j=0}^{n-1} y_j$ such that $Y_{n-1} = z_{n-1}G$.

For each adjacent pair (U_i, U_{i+1}) , the Schnorr key-generation procedure gives the parties additive signing-key shares x_i and x_{i+1} under the shared public key $\text{pk}_i = (x_i + x_{i+1})G$. The corresponding shared Schnorr key is used only for the lock on that payment-channel edge.

B. Lock Generation

Consider the lock between U_i and U_{i+1} . The shared path-dependent point for this lock is Y_i . The parties sample $r_{i,0}, r_{i,1} \leftarrow \mathbb{Z}_q$ and compute $R_{i,0} = r_{i,0}G$ and $R_{i,1} = r_{i,1}G$.

As in the original Schnorr-AMHL construction, a commitment-and-proof exchange is used to prevent either party from choosing its nonce contribution after observing the other party's value.

The effective nonce commitment for the lock is $R_i = R_{i,0} + R_{i,1} + Y_i$, and the corresponding Schnorr challenge is $e_i = H(\text{pk}_i \parallel R_i \parallel m_i)$.

The downstream party first computes the partial response $a_i = r_{i,1} + e_i x_{i+1} \bmod q$. After verifying this partial response, the upstream party computes $\hat{s}_i = a_i + r_{i,0} + e_i x_i \bmod q$.

Therefore, $\hat{s}_i G = R_{i,0} + R_{i,1} + e_i \text{pk}_i = R_i - Y_i + e_i \text{pk}_i$.

The lock is $\ell_i = (m_i, \text{pk}_i)$. The upstream party stores the right state $s_i^R = \hat{s}_i$, while the downstream party stores the left state $s_{i+1}^L = (R_i, \hat{s}_i)$.

Importantly, $(R_i, \hat{s}_i)$ is not yet a valid Schnorr opening. Compared with the Schnorr verification equation, the response lacks the discrete logarithm of the path-dependent point Y_i .

Let z_i satisfy $Y_i = z_i G$. Then $k_i = (R_i, \widehat{s}_i + z_i)$ is a valid opening key because $(\widehat{s}_i + z_i)G = R_i - Y_i + e_i \text{pk}_i + Y_i = R_i + e_i \text{pk}_i$.

For the final lock, the receiver knows $z_{n-1} = \sum_{j=0}^{n-1} y_j$ from the setup phase and can therefore complete the final incomplete response and initiate the backward release.

C. Verification

On input $\ell_i = (m_i, \text{pk}_i)$ and $k_i = (R_i, s_i)$, the verification algorithm computes $e_i = H(\text{pk}_i \parallel R_i \parallel m_i)$ and accepts if and only if $s_i G = R_i + e_i \text{pk}_i$.

Thus, verification is identical to ordinary Schnorr verification. The path-dependent information affects only whether a valid opening can be constructed, not how that opening is verified.

D. Backward Release

Consider an intermediate node U_i . Its setup state is $s_i^I = (Y_{i-1}, Y_i, y_i)$, its left state is $s_i^L = (R_{i-1}, \widehat{s}_{i-1})$, and its right state is $s_i^R = \widehat{s}_i$.

Suppose that U_i receives a valid opening $k_i = (R_i, s_i)$ for its right lock. It computes $s_{i-1} = \widehat{s}_{i-1} + s_i - (\widehat{s}_i + y_i) \bmod q$ and outputs $k_{i-1} = (R_{i-1}, s_{i-1})$.

To see why this produces a valid opening, write $s_i = \widehat{s}_i + z_i$ with $Y_i = z_i G$. Since $Y_i = Y_{i-1} + y_i G$, we obtain $z_{i-1} = z_i - y_i$ and $Y_{i-1} = z_{i-1} G$.

Substituting into the release equation gives $s_{i-1} = \widehat{s}_{i-1} + \widehat{s}_i + z_i - (\widehat{s}_i + y_i) = \widehat{s}_{i-1} + z_{i-1}$.

Therefore, $s_{i-1} G = \widehat{s}_{i-1} G + Y_{i-1} = R_{i-1} + e_{i-1} \text{pk}_{i-1}$.

Hence, $k_{i-1} = (R_{i-1}, s_{i-1})$ is a valid opening of the incoming lock.

This recursive transformation is the essential path-dependent property of Schnorr-AMHL: an intermediary can derive a valid opening of its incoming lock only after obtaining a valid opening of its outgoing lock. Consequently, release proceeds backward along the established payment path.

E. Use in DecouplePay

Schnorr-AMHL supplies the path-dependent lock/release mechanism on the payer-to-final-intermediary route. Its purpose in DecouplePay is independent of the payee-side redemption mechanism: AMHL ensures path consistency and prevents wormhole-style bypass of honest routed intermediaries.

DecouplePay additionally associates every routed payment with a delayed blind-signature response ς . Let $r = H(\varsigma)$. The routed payment requires the independent condition $H(\varsigma) = r$ in addition to the Schnorr-AMHL opening condition.

During a successful release, each routed intermediary therefore obtains the AMHL opening information needed to derive the opening of its incoming lock, while the same value ς is propagated backward toward the payer. Once the release reaches the payer, ς can be used to derive the authorization required for the separate payee-side redemption.

The two mechanisms consequently have distinct roles: Schnorr-AMHL binds the release to the routed path, whereas the hash condition on ς binds successful routed release to the delayed redemption authorization.

II. EVOLUTION OF THE FINAL-INTERMEDIARY/PAYEE CHANNEL

This section illustrates the state evolution of the persistent payment channel between the final intermediary F and the payee P . The purpose of this channel is different from that of the routed AMHL channels. Rather than forwarding the multi-hop payment, the F - P channel provides the enforceable payee-side transfer corresponding to a successfully released routed payment.

The channel follows a standard revocable commitment structure. The parties lock their deposits in a shared funding output and maintain successive off-chain commitment states. When the channel advances, fresh commitment transactions replace the previous state, and the obsolete state becomes subject to the underlying revocation mechanism.

A. Prepared Payment State

Suppose that the current settled balances of F and P are bal_F and bal_P , with total channel value $V = \text{bal}_F + \text{bal}_P$.

To prepare a DecouplePay payment of fixed value α , the parties construct a successor commitment state that reserves α from F 's balance in a dedicated redemption output. The ordinary balance outputs therefore represent $\text{bal}_F - \alpha$ for F and bal_P for P , while the dedicated redemption output carries α .

Hence, the total channel value remains unchanged: $(\text{bal}_F - \alpha) + \text{bal}_P + \alpha = V$.

The dedicated output represents a pending payee-side claim. It is not yet part of P 's ordinary settled balance. Instead, it can be exercised only under the redemption conditions associated with the current prepared state.

B. Redemption and Timeout

Each prepared state defines the redemption transaction or transactions needed to spend the dedicated redemption output. In the two-sided RSMC example used in the main paper, the two asymmetric commitment views induce two alternative redemption transactions. Only one commitment view can be confirmed on-chain, so these transactions represent alternative enforcement paths for the same logical payment rather than independent payments.

A successful DecouplePay routed release provides the payer with the delayed blind-signature response needed to derive F 's authorization for the corresponding redemption transaction. The payee can then use the resulting authorization to enforce the claim represented by the dedicated output.

If the routed payment is not successfully completed, the dedicated output is instead resolved through its timeout branch, allowing the reserved value to return to F according to the channel's timeout rules.

C. Channel-State Evolution

After successful settlement, the reserved value can be incorporated into P 's ordinary balance in a subsequent channel update. The channel therefore moves from a state containing a

pending redemption output to a later state in which the completed payment is reflected in the ordinary balance distribution.

For successive payments, the same pattern repeats. Each new prepared state reserves the next value α in a fresh redemption output, while already completed payments are reflected in the updated balances.

When the channel advances, the previous commitment state is revoked. Accordingly, redemption descriptors associated only with a superseded state are obsolete for safe settlement. If an obsolete commitment transaction is broadcast, the underlying revocation mechanism protects the counterparty according to the channel's stale-state penalty rules.

D. Illustrated Evolution

Fig. 1 illustrates the evolution of the channel between F and P . The figure shows the initial state, an updated state, and a later i -th state. Each state contains two asymmetric commitment views together with the corresponding ordinary balance outputs, dedicated redemption output, redemption transactions, delayed spends, and revocation paths.

The dedicated redemption output evolves together with the channel version. Consequently, a valid redemption descriptor is associated with a particular prepared state, while descriptors belonging only to revoked states are no longer valid for safe settlement.

III. RELATION TO THE MAIN DECOUPLEPAY CONSTRUCTION

The two components described in this document are complementary.

The Schnorr-AMHL mechanism operates exclusively on the payer-to-final-intermediary routed path. It ensures that release is path-dependent and that an intermediary can obtain the opening of its incoming lock only after its outgoing lock has been released.

The final-intermediary/payee channel operates outside this routed lock chain. It holds the value that may later be claimed by the payee and provides the fallback enforcement structure required for unilateral settlement.

DecouplePay connects these otherwise separate components through the delayed blind-signature response ς . During setup, the routed payment is bound to $r = H(\varsigma)$. A successful routed release reveals ς to the payer. The payer then uses this response to derive the authorization required for the corresponding payee-side redemption.

This separation allows the routed payment path and its routing-relevant information to remain independent of the later payee-side redemption, while still cryptographically ensuring that successful routed release supplies the authorization required for settlement.

SCOPE

This document is intended as supplementary technical material. It does not restate the complete DecouplePay protocol or its security proof. In particular, the formal leakage model,

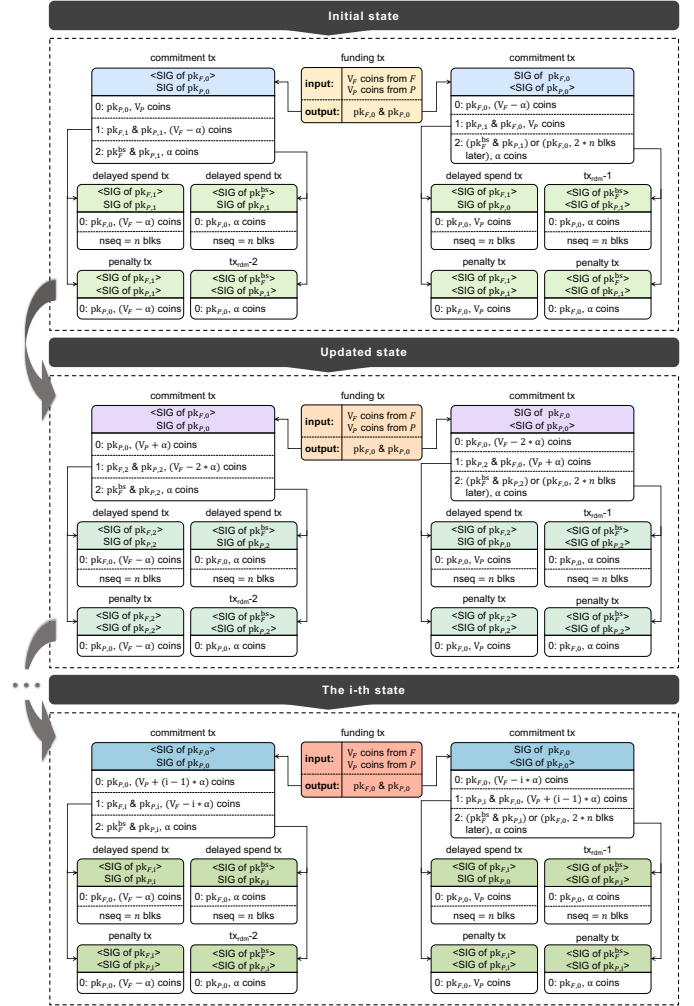


Fig. 1. Evolution of the payment channel between the final intermediary F and the payee P . A prepared state reserves the fixed payment value α in a dedicated redemption output. After settlement, completed payments are incorporated into the ordinary channel balances, while obsolete commitment states remain subject to the underlying revocation mechanism.

leakage-equivalent anonymity set, ideal functionality, UC security analysis, blind-signature response-binding construction, and performance evaluation are given in the main paper.

The purpose of this document is instead to make explicit two concrete components that are useful when reproducing or implementing the protocol: the algebraic operation of the Schnorr-AMHL routed locking mechanism and the state evolution of the final-intermediary/payee payment channel.

REFERENCES

- [1] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2019.